

1. Core FRP State Management Architecture

The implementation is there to show functional reactive programming through immutable state accumulation mainly using RXJS Scan Operator. This is done so to maintain referential transparency using only one single state object flowing immutably through observable pipelines.

The reducerOState function is there to process the five action types through pure pattern matching, this is done so to ensure that its deterministic, and to eliminate race condition and to ensure identical outputs for identical inputs – a core FP principles

Designer's note: custom Function LePiping is a utility to mainly enable the ticker to compose 11 pure state transformation in a readable pipeline and to avoid the need for a nested function.

2. Observable usage

Stream Architecture: ThrottleTime is used for trade actions, this is done so to avoid “cheesing” or exploitation while ensure that the action will execute immediately on valid input

Stream Composition: Five input sources is merged through a type safe const assertions, this is done so to ensure single responsibility per input channel while maintaining the FRP pattern

3. State Management

The in-game logic is done so to maintain strict referential transparency, through pure functions and immutable data structures. For example, the collision systems

Bounce system designs: Seed based deterministic randomness is used (e.g. bounceBird) returns 2 things that are the modified bird state and next seed, this is done so to ensure behavior is essential for testing and debugging

Trade-off analysis: Pure mathematical functions (pureMathAbs and PureMathMax) are done so to replace referential transparency; this increases code verbosity. So complete functional purity is more important than minor readability costs

4. Ghost Replay System

Ghost positioning systems are captured using pure state transformation and are captured through pure state transformation using frame-based sampling. The function `ghostPosFinder` uses frame interpolation to handle the missing data points

Performance considerations: Sampling balances the replay fidelity with the efficiency of the resources, this is done so to prevent memory pressure while also maintaining meaningful ghost visualization

5. Pipeline Architecture and Side Effect Containment

The ticker function demonstrates advanced functional composition through `lePiping`, applying physics, collision detection, scoring, and win conditions in a single transformation chain. Each transformation maintains immutability while composition ensures predictable execution order.

Critical Design Decision: All side effects are isolated to the render function, maintaining FRP purity throughout game logic. The functional approach clears and redraws elements each frame, eliminating synchronization issues between game state and visual representation.

Advanced Stuff

- **Invincibility Power Ups**

The implementation is dual activated (two ways of activating it) that is using frame – based timings). When the bird has collided with something, invincibility (120 frames) is activated post damage, while the power up version of the invincibility is activated for 180 frames, the system is made so that it integrated the countdown timers directly into the bird state through `updatedBirdPhysics` thus maintaining functional purity while providing visual feedback via the opacity modulation

- **Dynamic Trading Mechanism**

The score to life trading system is introduced to help player be more strategic with resource management, through the function `logicOfTrades` validation and `tradeApplier` execution, the player will balance the immediate safety against long term scoring potential, thus create meaning decisions in gameplay, `throttleTime` is also used to prevent exploitation.

- **Power Ups Collection system:**

Physics based power up types are also introduced, mainly in the movement and functional state management. The `powPartitioner` function utilizes `reduce` to separate the collected from active power ups, visual feedback is also introduced by including type specific coloring and dynamic opacity changes.